Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: every later step adds nodes and branches onto one LangGraph graph. Before any of that, this step proves the mechanics work on the smallest graph that does something real: a typed `State`, two nodes (plain functions that take the state and return field updates), plain edges connecting them, and one real model call inside a node. **If not:** when Step 3's branching misbehaves, you won't know if the problem is the branch or your understanding of how state flows.

After this step the project runs: `python main.py` answers questions through a 2-node graph, and prints which nodes ran.

The shared files

These are the same files as Projects 1 and 2 — Doc01's config, errors and logger, and Project 1's prompt loader — with only the package name changed to `src`.

Story — `exceptions.py`, `config.py`, `utils/logger.py`: these are Doc01's familiar foundations: a named `MissingConfigError`, a `Config` that loads `.env` and fails loudly the moment a required setting is missing, and one `get_logger(name)` shared by every file. **If not:** a missing key would surface deep inside a graph node as a confusing error, far from where the real cause is.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env. (Doc01's file.)
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing. (Doc01's file.)
import os
from dataclasses import dataclass
from dotenv import load_dotenv
```

```
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
# file name and the level on every line. (Doc01's file.)
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        logger.setLevel(level)

    return logger
```

Also create empty `__init__.py` files in `src/utils/` and `src/models/`.

Back to this step · Hint 1 · Hint 2 · Solution

Solution — state, nodes and graph in their own files, with a path log

Story — `models/llm.py`: this is the one place that builds the `ChatOpenAI` model. Graph nodes only ever receive the state (that's the whole contract LangGraph nodes follow), so each node calls `get_model()` for itself rather than being handed a model object. **If not:** every node would build its own `ChatOpenAI` with its own settings, scattering the model configuration across the codebase.

```
# src/models/llm.py
# get_model(): the one ChatOpenAI every node uses. ChatOpenAI reads
# OPENAI_API_KEY from the environment, filled by load_config().
from langchain_openai import ChatOpenAI

def get_model():
    return ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

Story — `state.py`: this `TypedDict` is the graph's whole public surface — every field every node can read or write (Doc09's pattern). `path` is annotated with `operator.add`, so when a node returns `{"path": ["answer_node"]}` LangGraph *appends* it to the existing list instead of replacing it — the graph ends up keeping its own log of what ran. `start_state()` gives every field a known starting value, so no node ever reads an unset field. **If not:** field names would drift between nodes, and you'd need print statements to see the route taken.

```
# src/state.py
# DocState: everything any node can read or write. `path` adds each node's
# entries to the list (operator.add), so it logs the nodes that ran.
# start_state(task) gives every field a starting value.
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    task: str
    path: Annotated[list[str], operator.add]
    answer: str

def start_state(task):
    return {"task": task, "path": [], "answer": ""}
```

Story — `prompts/general.txt` + `utils/prompts.py`: the prompt text lives outside the code in a `.txt` file, loaded by name through Project 1's `load_prompt()` loader (folder changed to

`src/prompts`). **If not:** the wording would be buried inside a node function, and tweaking it would mean a code change.

`src/prompts/general.txt`

```
Answer this question from general knowledge, in at most 2 sentences.

Question: {task}
```

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live
# in text files instead of Python strings.

def load_prompt(name):
    path = "src/prompts" + "/" + name + ".txt"
    with open(path) as prompt_file:
        return prompt_file.read().strip()
```

Story — `nodes.py`: each node function takes the state and returns only a dict of the fields it changes, never the whole state (Doc09's rule). `answer_node` wraps a Doc05 LCEL chain inside a node and labels its answer as coming from general knowledge, because later steps add answers grounded in your actual documents. This file doesn't change again after this step — later steps add *new* node files instead. **If not:** a node returning the whole state could overwrite fields it never meant to touch.

```
# src/nodes.py
# Small non-agent nodes: clean_task() tidies the question once;
# answer_node() answers general questions, labelled as general knowledge.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def clean_task(state):
    return {"task": state["task"].strip(), "path": ["clean_task"]}

def answer_node(state):
    prompt = ChatPromptTemplate.from_template(load_prompt("general"))
    chain = prompt | get_model() | StrOutputParser()
    reply = chain.invoke({"task": state["task"]})
    answer = reply + "\n(general knowledge — not from your documents)"
    return {"answer": answer, "path": ["answer_node"]}
```

Story — `graph.py`: `build_graph()` creates the `StateGraph`, adds the nodes, wires the edges, and returns the compiled graph — a graph object `main.py` or a test can run with `.invoke()`. Keeping this in a function (not top-level script code) means `main.py`, the tests, and later scripts can all build the exact same graph. **If not:** the graph would exist only while one script runs top to bottom, with no reusable way to build it elsewhere.

```
# src/graph.py
# build_graph(): two nodes in a straight line — clean_task, then
# answer_node — compiled into a graph.
from langgraph.graph import END, START, StateGraph

from src.nodes import answer_node, clean_task
from src.state import DocState

def build_graph():
    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)

    builder.add_edge(START, "clean_task")
    builder.add_edge("clean_task", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()
```

Story — `main.py`: this wires Step 1 together in order: load config, build the graph once, then run a question loop that invokes the graph and prints both the `path` it took and the final answer — so you can see the graph's internal routing, not just its output. **If not:** you couldn't see the graph working, only its final answer with no visibility into how it got there.

```
# main.py
# The file you run: python main.py
# Loads the settings, builds the graph, then answers questions in a loop,
# printing the path each run took.
import sys

import openai

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.state import start_state
from src.utils.logger import get_logger

def main():
    try:
```

```
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    graph = build_graph()
    print("DocuMind — ask a question, or /quit to exit.")

    while True:
        task = input("You: ").strip()
        if task == "/quit":
            break
        if task == "":
            continue

        try:
            result = graph.invoke(start_state(task))
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)

        print(f"  path: {' -> '.join(result['path'])}")
        print(f"Answer: {result['answer']}")

if __name__ == "__main__":
    main()
```

Run it: `python main.py`

```
DocuMind — ask a question, or /quit to exit.
You: What is LangGraph?
  path: clean_task -> answer_node
Answer: LangGraph is a library for building stateful, multi-step agent
workflows as graphs of nodes and edges.
(general knowledge — not from your documents)
You: /quit
```

(The answer is wrapped here to fit the page; the wording varies.)

Trace it by hand before moving on: after `clean_task`, the state is `{"task": "What is LangGraph?", "path": ["clean_task"], "answer": ""}`. After `answer_node`, `path` is `["clean_task", "answer_node"]` and `answer` holds the reply.

The Step 1 tests

Story — `tests/unit/test_config.py`: this is the fast, no-cost safety net for the settings code — the same three checks as Doc01's `test_config.py`: a missing setting, an empty one, and a config that loads, all with no API call. It uses a made-up setting name, so your real `.env` is never touched. **If not:** a broken `.env` would first show up as a confusing OpenAI error deep in a graph node.

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env file is never
# touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "DOCUMIND_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
```

```
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Story — `tests/unit/test_nodes.py`: `clean_task()` and `start_state()` are plain functions with no model call inside them, so they're checked with hand-made input — no model, no graph, instant and free. **If not:** a node that forgot to set its `path` entry would only show up as a strange, incomplete path in a real run.

```
# tests/unit/test_nodes.py
# Checks the small nodes and the starting state with hand-made input.
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_nodes
from src.nodes import clean_task
from src.state import start_state

def test_clean_task():
    update = clean_task({"task": "  What is a refund?  "})
    result = "OK"
    if update["task"] != "What is a refund?":
        result = "WRONG"
    if update["path"] != ["clean_task"]:
        result = "WRONG"
    print(f"case 1 (clean_task tidies the task) -> {result}")

def test_start_state():
    state = start_state("hello")
    result = "OK"
    if state["task"] != "hello" or state["path"] != []:
        result = "WRONG"
    if state["answer"] != "":
        result = "WRONG"
    print(f"case 2 (start_state starts empty)   -> {result}")

if __name__ == "__main__":
    test_clean_task()
    test_start_state()
```

Story — `tests/integration/test_live_graph.py`: this holds the checks that genuinely need the real model and a real graph run. It starts with one case — a general question goes through both nodes and comes back with a real answer — and every later step adds its own cases to this same growing file. **If not:** "the graph works" would mean "it worked the one time I tried it," with nothing to re-run after a change.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_graph
from src.config import load_config
from src.graph import build_graph
from src.state import start_state

def run(graph, thread_id, task):
    config = {"configurable": {"thread_id": thread_id}}
    return graph.invoke(start_state(task), config)

def show(case, passed, outcome):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {outcome.get('path')}")

def test_general_question(graph):
    outcome = run(graph, "t1", "What is the capital of France?")
    passed = "Paris" in outcome["answer"] and "answer_node" in outcome["path"]
    show("case 1 (general question)", passed, outcome)

if __name__ == "__main__":
    load_config()
    graph = build_graph()
    test_general_question(graph)
```

Create the empty folders `tests/unit/` and `tests/integration/` too. Tests run as modules from the repo root, so they need no `__init__.py`.

Every test prints `OK` or `WRONG` — you never have to judge the output yourself. `WRONG` means: read that test function and the code it calls.

Run the tests (from the repo root):

```
python -m tests.unit.test_config
python -m tests.unit.test_nodes
python -m tests.integration.test_live_graph
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
```

```
case 3 (config loads)    -> OK
```

```
case 1 (clean_task tidies the task) -> OK
case 2 (start_state starts empty)   -> OK
```

```
case 1 (general question) -> OK: ['clean_task', 'answer_node']
```